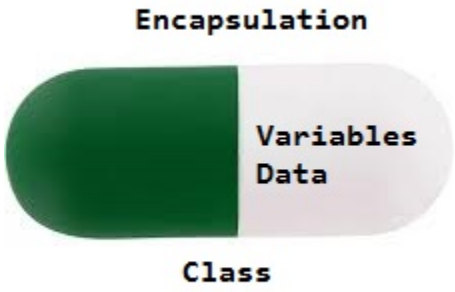
Encapsulation in Java

Last Updated : 04 Oct, 2024

Encapsulation in Java is a fundamental concept in object-oriented programming (OOP) that refers to the bundling of data and methods that operate on that data within a single unit, which is called a class in Java. Java Encapsulation is a way of hiding the implementation details of a class from outside access and only exposing a public interface that can be used to interact with the class.

In Java, encapsulation is achieved by declaring the instance variables of a class as private, which means they can only be accessed within the class. To allow outside access to the instance variables, public methods called getters and setters are defined, which are used to retrieve and modify the values of the instance variables, respectively. By using getters and setters, the class can enforce its own data validation rules and ensure that its internal state remains consistent.



Implementation of Java Encapsulation

Encapsulation ensures that the internal workings of a class are hidden, promoting modular code. To explore how encapsulation works and learn best practices for applying it in large applications, the Java Programming Course provides detailed lessons and hands-on coding tasks

Below is the example with Java Encapsulation:

Java

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```java
// Java Program to demonstrate
// Java Encapsulation

// Person Class
class Person {
    // Encapsulating the name and age
    // only approachable and used using
    // methods defined
    private String name;
    private int age;

    public String getName() { return name; }

    public void setName(String name) { this.name =
name; }

    public int getAge() { return age; }

    public void setAge(int age) { this.age = age; }
}

// Driver Class
public class Main {
    // main function
    public static void main(String[] args)
    {
        // person object created
        Person person = new Person();
        person.setName("John");
        person.setAge(30);

        // Using methods to get the values from the
        // variables
        System.out.println("Name: " +
person.getName());
        System.out.println("Age: " + person.getAge());
    }
}
```

Output

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Name: John
Age: 30
```

Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates. Another way to think about encapsulation is, that it is a protective shield that prevents the data from being accessed by the code outside this shield.

- Technically in encapsulation, the variables or data of a class is hidden from any other class and can be accessed only through any member function of its own class in which it is declared.
- As in encapsulation, the data in a class is hidden from other classes using the data hiding concept which is achieved by making the members or methods of a class private, and the class is exposed to the end-user or the world without providing any details behind implementation using the abstraction concept, so it is also known as a **combination of data-hiding and abstraction**.
- Encapsulation can be achieved by Declaring all the variables in the class as private and writing public methods in the class to set and get the values of variables.
- It is more defined with the setter and getter method.

Advantages of Encapsulation

- **Data Hiding:** it is a way of restricting the access of our data members by hiding the implementation details. Encapsulation also provides a way for data hiding. The user will have no idea about the inner implementation of the class. It will not be visible to the user how the class is storing values in the variables. The user will only know that we are passing the values to a setter method and variables are getting initialized with that value.
- **Increased Flexibility:** We can make the variables of the class read-only or write-only depending on our requirements. If we wish to make the variables read-only then we have to omit the setter methods like setName(), setAge(), etc. from the above program or if we wish to make the variables write-only then we have to omit the get methods like getName(), getAge(), etc. from the above program

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- **Reusability:** Encapsulation also improves the re-usability and is easy to change with new requirements.
- **Testing code is easy:** Encapsulated code is easy to test for unit testing.
- **Freedom to programmer in implementing the details of the system:** This is one of the major advantage of encapsulation that it gives the programmer freedom in implementing the details of a system. The only constraint on the programmer is to maintain the abstract interface that outsiders see.

> ***For example:*** *The Programmer of the edit menu code in a text-editor GUI might at first, implement the cut and paste operations by copying actual screen images in and out of an external buffer. Later, he/she may be dissatisfied with this implementation, since it does not allow compact storage of the selection, and it does not distinguish text and graphic objects. If the programmer has designed the cut-and-paste interface with encapsulation in mind, switching the underlying implementation to one that stores text as text and graphic objects in an appropriate compact format should not cause any problems to functions that need to interface with this GUI. Thus encapsulation yields adaptability, for it allows the implementation details of parts of a program to change without adversely affecting other parts.*

Disadvantages of Encapsulation in Java

- Can lead to increased complexity, especially if not used properly.
- Can make it more difficult to understand how the system works.
- May limit the flexibility of the implementation.

Examples Showing Data Encapulation in Java

Example 1:

Below is the implementation of the above topic:

Java

```
// Java Program to demonstrate
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```java
// Java Encapsulation

// fields to calculate area
class Area {
    int length;
    int breadth;

    // constructor to initialize values
    Area(int length, int breadth)
    {
        this.length = length;
        this.breadth = breadth;
    }

    // method to calculate area
    public void getArea()
    {
        int area = length * breadth;
        System.out.println("Area: " + area);
    }
}

class Main {
    public static void main(String[] args)
    {

        Area rectangle = new Area(2, 16);
        rectangle.getArea();
    }
}
```

Output

```
Area: 32
```

Example 2:

The program to access variables of the class EncapsulateDemo is shown below:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Java

```java
// Java program to demonstrate
// Java encapsulation

class Encapsulate {
    // private variables declared
    // these can only be accessed by
    // public methods of class
    private String geekName;
    private int geekRoll;
    private int geekAge;

    // get method for age to access
    // private variable geekAge
    public int getAge() { return geekAge; }

    // get method for name to access
    // private variable geekName
    public String getName() { return geekName; }

    // get method for roll to access
    // private variable geekRoll
    public int getRoll() { return geekRoll; }

    // set method for age to access
    // private variable geekage
    public void setAge(int newAge) { geekAge = newAge;
}

    // set method for name to access
    // private variable geekName
    public void setName(String newName)
    {
        geekName = newName;
    }

    // set method for roll to access
    // private variable geekRoll
    public void setRoll(int newRoll) { geekRoll =
newRoll; }
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
}

public class TestEncapsulation {
    public static void main(String[] args)
    {
        Encapsulate obj = new Encapsulate();

        // setting values of the variables
        obj.setName("Harsh");
        obj.setAge(19);
        obj.setRoll(51);

        // Displaying values of the variables
        System.out.println("Geek's name: " +
obj.getName());
        System.out.println("Geek's age: " +
obj.getAge());
        System.out.println("Geek's roll: " +
obj.getRoll());

        // Direct access of geekRoll is not possible
        // due to encapsulation
        // System.out.println("Geek's roll: " +
        // obj.geekName);
    }
}
```

Output

```
Geek's name: Harsh
Geek's age: 19
Geek's roll: 51
```

Example 3:

In the above program, the class Encapsulate is encapsulated as the variables are declared private. The get methods like getAge(), getName(), and getRoll() are set as public, these methods are used to access these variables. The setter methods

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

like setName(), setAge(), setRoll() are also declared as public and are used to set the values of the variables.

Below is the implementation of the defined example:

Java

```java
// Java Program to demonstrate
// Java Encapsulation

class Name {
    // Private is using to hide the data
    private int age;

    // getter
    public int getAge() { return age; }

    // setter
    public void setAge(int age) { this.age = age; }
}

// Driver Class
class GFG {
    // main function
    public static void main(String[] args)
    {
        Name n1 = new Name();
        n1.setAge(19);
        System.out.println("The age of the person is: "
                           + n1.getAge());
    }
}
```

Output

```
The age of the person is: 19
```

Example 4:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Below is the implementation of the Java Encapsulation:

Java

```java
// Java Program to demonstrate
// Java Encapsulation

class Account {
    // private data members to hide the data
    private long acc_no;
    private String name, email;
    private float amount;
    // public getter and setter methods
    public long getAcc_no() { return acc_no; }
    public void setAcc_no(long acc_no)
    {
        this.acc_no = acc_no;
    }
    public String getName() { return name; }
    public void setName(String name) { this.name = name;
}
    public String getEmail() { return email; }
    public void setEmail(String email)
    {
        this.email = email;
    }
    public float getAmount() { return amount; }
    public void setAmount(float amount)
    {
        this.amount = amount;
    }
}

// Driver Class
public class GFG {
      // main function
    public static void main(String[] args)
    {
        // creating instance of Account class
        Account acc = new Account();
        // setting values through setter methods
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
37          acc.setAcc_no(90482098491L);
38          acc.setName("ABC");
39          acc.setEmail("abc@gmail.com");
40          acc.setAmount(100000f);
41          // getting values through getter methods
42          System.out.println(
43              acc.getAcc_no() + " " + acc.getName() + " "
44              + acc.getEmail() + " " + acc.getAmount());
45      }
46  }
```

Output

```
90482098491 ABC abc@gmail.com 100000.0
```

Encapsulation with Java

Visit Course ↗

Comment

More info

Next Article

Inheritance in Java

Similar Reads

Java Tutorial

Java is one of the most popular and widely used programming languages which is used to develop mobile apps, desktop apps, web apps, web servers, games, and enterprise-level systems. Java was...

4 min read

Java Overview

Java Basics

Java Flow Control

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Java Methods

Java Arrays

Java Strings

Java OOPs Concepts

Object Oriented Programming (OOPs) Concept in Java

As the name suggests, Object-Oriented Programming or Java OOPs concept refers to languages that use objects in programming, they use objects as a primary source to implement what is to happen in th...

12 min read

Classes and Objects in Java

In Java, classes and objects are basic concepts of Object Oriented Programming (OOPs) that are used to represent real-world concepts and entities. The class represents a group of objects having similar...

11 min read

Java Constructors

Java constructors or constructors in Java is a terminology used to construct something in our programs. A constructor in Java is a special method that is used to initialize objects. The constructor is called when a...

9 min read

Object Class in Java

Object class is present in java.lang package. Every class in Java is directly or indirectly derived from the Object class. If a class does not extend any other class then it is a direct child class of Object and if...

7 min read

Abstraction in Java

Abstraction in Java is the process in which we only show essential details/functionality to the user. The non-essential implementation details are not displayed to the user. In this article, we will learn about...

11 min read

Encapsulation in Java

Encapsulation in Java is a fundamental concept in object-oriented programming (OOP) that refers to the bundling of data and methods that operate on that data within a single unit, which is called a class in...

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

8 min read

Inheritance in Java

Java, Inheritance is an important pillar of OOP(Object-Oriented Programming). It is the mechanism in Java by which one class is allowed to inherit the features(fields and methods) of another class. In Java,...

13 min read

Polymorphism in Java

The word 'polymorphism' means 'having many forms'. In simple words, we can define Java Polymorphism as the ability of a message to be displayed in more than one form. In this article, we will...

6 min read

Method Overloading in Java

In Java, Method Overloading allows different methods to have the same name, but different signatures where the signature can differ by the number of input parameters or type of input parameters, or a...

9 min read

Overriding in Java

In Java, Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes. When a method in a...

13 min read

Packages In Java

Package in Java is a mechanism to encapsulate a group of classes, sub packages and interfaces. Packages are used for: Preventing naming conflicts. For example there can be two classes with name...

8 min read

Java Interfaces

Java Collections

Java Exception Handling

Java Multithreading

Java File Handling

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Java Streams and Lambda Expressions

Java IO

Java Synchronization

Java Regex

Java Networking

JDBC

Java Memory Allocation

Java Interview Questions

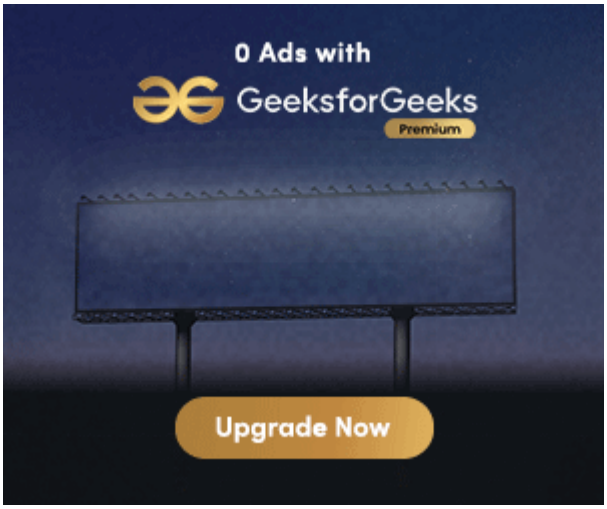
Java Practice Problems

Java Projects

Article Tags : Java Java-Object Oriented

Practice Tags : Java

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Explore our developer-friendly HTML to PDF API
Printed using PDFCrowd
HTML to PDF